Problem List Submit Premium

Description | Editorial | Solutions | Submissions

1603. Design Parking System

Solved

Easy | Topics | Companies | Hint

Design a parking system for a parking lot. The parking lot has three kinds of parking spaces: big, medium, and small, with a fixed number of slots for each size.

Implement the `ParkingSystem` class:

- `ParkingSystem(int big, int medium, int small)` Initializes object of the `ParkingSystem` class. The number of slots for each parking space are given as part of the constructor.
- `bool addCar(int carType)` Checks whether there is a parking space of `carType` for the car that wants to get into the parking lot. `carType` can be of three kinds: big, medium, or small, which are represented by `1`, `2`, and `3` respectively. **A car can only park in a parking space of its** `carType`. If there is no space available, return `false`, else park the car in that size space and return `true`.

Example 1:

```
Input
["ParkingSystem", "addCar", "addCar", "addCar", "addCar"]
[[1, 1, 0], [1], [2], [3], [1]]
Output
[null, true, true, false, false]

Explanation
ParkingSystem parkingSystem = new ParkingSystem(1, 1, 0);
parkingSystem.addCar(1); // return true because there is 1 available slot for a big car
parkingSystem.addCar(2); // return true because there is 1 available slot for a medium car
parkingSystem.addCar(3); // return false because there is no available slot for a small car
parkingSystem.addCar(1); // return false because there is no available slot for a big car. It is already occupied.
```

2.1K | 70 | 31 Online

Code | Testcase | Test Result

Java | Auto

```java
class ParkingSystem {

    int big;
    int medium;
    int small;

    public ParkingSystem(int big, int medium, int
        this.big = big;
        this.medium = medium;
        this.small = small;
    }

    public boolean addCar(int carType) {
        if (carType == 1) {
            if (big > 0) {
                big--;
                return true;
            }
        }

        else if (carType == 2) {
            if (medium > 0) {
                medium--;
                return true;
            }
        }

        else {
            if (small > 0) {
                small--;
                return true;
            }
        }
        return false;
    }
}

/**
 * Your ParkingSystem object will be instantiated and called as such:
 * ParkingSystem obj = new ParkingSystem(big, medium, small);
 * boolean param_1 = obj.addCar(carType);
```

Saved | Ln 9, Col 30

JANIISH R

Access all features with our Premium subscription!

My Lists | Notebook | Progress | Points

- Try New Features
- Orders
- My Playgrounds
- Settings
- Appearance
- Sign Out

Problem List Submit Premium

Description | Editorial | Solutions | Accepted | Submissions

705. Design HashSet

Solved

Easy Topics Companies

Design a HashSet without using any built-in hash table libraries.

Implement `MyHashSet` class:

- `void add(key)` Inserts the value `key` into the HashSet.
- `bool contains(key)` Returns whether the value `key` exists in the HashSet or not.
- `void remove(key)` Removes the value `key` in the HashSet. If `key` does not exist in the HashSet, do nothing.

Example 1:

```
Input
["MyHashSet", "add", "add", "contains", "contains", "add", "contains", "remove",
"contains"]
[[], [1], [2], [1], [3], [2], [2], [2], [2]]
Output
[null, null, null, true, false, null, true, null, false]

Explanation
MyHashSet myHashSet = new MyHashSet();
myHashSet.add(1);      // set = [1]
myHashSet.add(2);      // set = [1, 2]
myHashSet.contains(1); // return True
myHashSet.contains(3); // return False, (not found)
myHashSet.add(2);      // set = [1, 2]
myHashSet.contains(2); // return True
myHashSet.remove(2);   // set = [1]
myHashSet.contains(2); // return False, (already removed)
```

Constraints:

4.1K 88 17 Online

Code | Testcase | Test Result

Java Auto

```java
class MyHashSet {
    private boolean[] mp;

    public MyHashSet() {
        mp = new boolean[1000001];
        Arrays.fill(mp, false);
    }

    public void add(int key) {
        mp[key] = true;
    }

    public void remove(int key) {
        mp[key] = false;
    }

    public boolean contains(int key) {
        return mp[key];
    }
}

/**
 * Your MyHashSet object will be instantiated and
 * MyHashSet obj = new MyHashSet();
 * obj.add(key);
 * obj.remove(key);
 * boolean param_3 = obj.contains(key);
 */
```

Saved Ln 13, Col 34

JANIISH R

Access all features with our Premium subscription!

My Lists Notebook Progress Points

- Try New Features
- Orders
- My Playgrounds
- Settings
- Appearance
- Sign Out

Problem List

Submit

Premium

Description | Editorial | Solutions | Accepted | Submissions

1396. Design Underground System

Solved

Medium | Topics | Companies | Hint

An underground railway system is keeping track of customer travel times between different stations. They are using this data to calculate the average time it takes to travel from one station to another.

Implement the `UndergroundSystem` class:

- `void checkIn(int id, string stationName, int t)`
  - A customer with a card ID equal to `id`, checks in at the station `stationName` at time `t`.
  - A customer can only be checked into one place at a time.
- `void checkOut(int id, string stationName, int t)`
  - A customer with a card ID equal to `id`, checks out from the station `stationName` at time `t`.
- `double getAverageTime(string startStation, string endStation)`
  - Returns the average time it takes to travel from `startStation` to `endStation`.
  - The average time is computed from all the previous traveling times from `startStation` to `endStation` that happened **directly**, meaning a check in at `startStation` followed by a check out from `endStation`.
  - The time it takes to travel from `startStation` to `endStation` **may be different** from the time it takes to travel from `endStation` to `startStation`.
  - There will be at least one customer that has traveled from `startStation` to `endStation` before `getAverageTime` is called.

You may assume all calls to the `checkIn` and `checkOut` methods are consistent. If a customer checks in at time t_1 then checks out at time t_2, then $t_1 < t_2$. All events happen in chronological order.

Example 1:

Input

["UndergroundSystem","checkIn","checkIn","checkIn","checkOut","checkOut","checkOut", "getAverageTime","getAverageTime","checkIn","getAverageTime","checkOut","getAverageT

3.6K | 75 | 12 Online

Code | Testcase | Test Result

Java | Auto

```java
class Passenger {
    int checkinTime;
    int checkoutTime;
    String checkinLocation;
    String checkoutLocation;

    public Passenger(String checkinLocation, int
        this.checkinLocation = checkinLocation;
        this.checkinTime = checkinTime;
    }

    void checkout(String checkoutLocation, int ch
        this.checkoutLocation = checkoutLocation;
        this.checkoutTime = checkoutTime;
    }

}

class Route {
    String startStation;
    String endStation;
    int totalNumberOfTrips;
    long totalTimeSpentInTrips;

    public Route(String startStation, String endS
        this.startStation = startStation;
        this.endStation = endStation;
    }

    double getAverageTime() {
        return (double) totalTimeSpentInTrips / totalNumberOfTrips;
    }

    void addTrip(int startTime, int endTime) {
        totalTimeSpentInTrips += endTime - startTime;
        totalNumberOfTrips++;
    }
}

class UndergroundSystem {
```

Saved | Ln 11, Col 1

JANIISH R

Access all features with our Premium subscription!

My Lists | Notebook | Progress | Points

- Try New Features
- Orders
- My Playgrounds
- Settings
- Appearance
- Sign Out

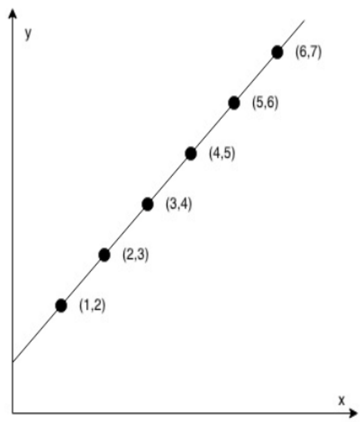
1232. Check If It Is a Straight Line

Solved

Easy | Topics | Companies | Hint

You are given an integer array `coordinates`, `coordinates[i] = [x, y]`, where `[x, y]` represents the coordinate of a point. Check if these points make a straight line in the XY plane.

Example 1:



Input: coordinates = [[1,2],[2,3],[3,4],[4,5],[5,6],[6,7]]
Output: true

Example 2:

2.7K | 79

Java | Auto

```java
class Solution {
    public boolean checkStraightLine(int[][]
        int x0 = coordinates[0][0], y0 = coordina
            x1 = coordinates[1][0], y1 = coordina
        int dx = x1 - x0, dy = y1 - y0;
        for (int[] co : coordinates) {
            int x = co[0], y = co[1];
            if (dx * (y - y1) != dy * (x - x1))
                return false;
        }
        return true;
    }
}
```

JANIISH R

Access all features with our Premium subscription!

My Lists | Notebook | Progress

Points

- Try New Features
- Orders
- My Playgrounds
- Settings
- Appearance
- Sign Out

Saved

Ln 1, Col 1

Problem List

Submit

Premium

Description | Editorial | Solutions | Accepted | Submissions

1114. Print in Order

Solved

Easy | Topics | Companies

Suppose we have a class:

```
public class Foo {
  public void first() { print("first"); }
  public void second() { print("second"); }
  public void third() { print("third"); }
}
```

The same instance of `Foo` will be passed to three different threads. Thread A will call `first()`, thread B will call `second()`, and thread C will call `third()`. Design a mechanism and modify the program to ensure that `second()` is executed after `first()`, and `third()` is executed after `second()`.

Note:

We do not know how the threads will be scheduled in the operating system, even though the numbers in the input seem to imply the ordering. The input format you see is mainly to ensure our tests' comprehensiveness.

Example 1:

```
Input: nums = [1,2,3]
Output: "firstsecondthird"
Explanation: There are three threads being fired asynchronously. The input [1,2,3]
means thread A calls first(), thread B calls second(), and thread C calls third().
"firstsecondthird" is the correct output.
```

Example 2:

```
Input: nums = [1,3,2]
Output: "firstsecondthird"
Explanation: The input [1,3,2] means thread A calls first(), thread B calls third(),
and thread C calls second(). "firstsecondthird" is the correct output.
```

1.6K | 48

Code | Testcase | Test Result | Code Sample

Java | Auto

```java
class Foo {
    boolean one;
    boolean two;

    public Foo() {
        one = false;
        two = false;
    }

    public synchronized void first(Runnable print
        printFirst.run();
        one = true;
        notifyAll();
    }

    public synchronized void second(Runnable prin
        while (!one)
            wait();
        printSecond.run();
        two = true;
        notifyAll();
    }

    public synchronized void third(Runnable print
        while (!two)
            wait();
        printThird.run();
    }
}
```

Saved

Ln 13, Col 21

JANIISH R

Access all features with our Premium subscription!

My Lists | Notebook | Progress

Points

Try New Features

Orders

My Playgrounds

Settings

Appearance

Sign Out

Problem List

Submit

Premium

Description | Editorial | Solutions | Submissions

242. Valid Anagram

Solved

Easy | Topics | Companies

Given two strings `s` and `t`, return `true` if `t` is an anagram of `s`, and `false` otherwise.

Example 1:

Input: `s = "anagram", t = "nagaram"`

Output: `true`

Example 2:

Input: `s = "rat", t = "car"`

Output: `false`

Constraints:

- `1 <= s.length, t.length <= 5 * 10`^{4}
- `s` and `t` consist of lowercase English letters.

Follow up: What if the inputs contain Unicode characters? How would you adapt your solution to such a case?

Seen this question in a real interview before? 1/6

14.6K | 346

334 Online

Code | Testcase | Test Result

Java | Auto

```java
class Solution {
    public boolean isAnagram(String s, String t)
        if (s.length() != t.length()) {
            return false;
        }

        int[] freq = new int[26];
        for (int i = 0; i < s.length(); i++) {
            freq[s.charAt(i) - 'a']++;
            freq[t.charAt(i) - 'a']--;
        }

        for (int i = 0; i < freq.length; i++) {
            if (freq[i] != 0) {
                return false;
            }
        }

        return true;
    }
}
```

Saved

Ln 1, Col 1

JANIISH R

Access all features with our Premium subscription!

My Lists | Notebook | Progress | Points

Try New Features

Orders

My Playgrounds

Settings

Appearance

Sign Out